ПОЯСНИТЕЛЬНЫЙ ОТЧЁТ

к лабораторной работе №4

«Обедающие философы: справедливое распределение ресурсов»

Вариант 6. Простыми словами о теории, реализации, проверках и результатах

Выполнил: Василенко Дмитрий Васильевич, группа 453503
Дисциплина: «Операционные среды и системное программирование»
Минск, 2026

Как читать этот отчёт. Это не формальный отчёт для сдачи, а разбор «своими словами». Сначала объясняется теория, потом — что именно требовалось сделать, затем — как устроена программа и почему она работает. В конце есть вопросы, которые могут задать на защите, и короткие ответы на них.

СОДЕРЖАНИЕ

1. САМАЯ КОРОТКАЯ СУТЬ 3
2. ТЕОРИЯ БЕЗ СЛОЖНЫХ СЛОВ 4
   2.1 Процесс и поток 4
   2.2 Гонка данных 4
   2.3 Критическая секция и мьютекс 4
   2.4 Атомарная заявка на две вилки 5
   2.5 Семафор и событие 5
   2.6 Тупик, голодание и справедливость 5
   2.7 Пропускная способность и время ожидания 5
   2.8 FIFO и SCAN 6
3. ЧТО ТРЕБОВАЛОСЬ ПО ЗАДАНИЮ 7
4. КАК УСТРОЕНА ПРОГРАММА 8
   4.1 Основные компоненты 8
   4.2 Жизненный цикл одной заявки 8
   4.3 Выдача вилок 9
   4.4 Тайм-аут 9
   4.5 Watchdog 9
   4.6 Инварианты 9
5. ЧТО БЫЛО СДЕЛАНО ПО ЭТАПАМ 11
   5.1 Этап 1. Проектирование 11
   5.2 Этап 2. Базовая реализация 11
   5.3 Этап 3. Синхронизация и тестирование 11
   5.4 Этап 4. Эксперименты и отчёт 12
6. КАК ЗАПУСКАТЬ И ПРОВЕРЯТЬ

. 13
7. РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ
. 14
7.1 Почему FIFO даёт около 58 оп/с 14
7.2 Почему SCAN быстрее 14
7.3 Почему у SCAN хуже максимальное ожидание 14
7.4 Почему при большем N выше пропускная способность 14
7.5 Графики 15
8. ЧТО БЫЛО БЫ, ЕСЛИ ДЕЛАТЬ С НУЛЯ
. 16
9. ВОПРОСЫ НА ЗАЩИТЕ
. 17
9.1 Зачем заявка сразу на две вилки? 17
9.2 Почему очередь центральная? 17
9.3 Зачем отдельный семафор на каждую заявку? 17
9.4 Почему сигнал не теряется? 17
9.5 Что такое overtake? 17
9.6 Почему FIFO справедливый? 17
9.7 Почему SCAN быстрее? 17
9.8 Почему у SCAN хуже максимальное ожидание? 17
9.9 Как проверяется отсутствие гонок? 17
9.10 Что будет при тайм-ауте? 17
9.11 Зачем watchdog, если есть тайм-ауты? 18
9.12 Почему нет отдельного потока-арбитра? 18
9.13 Почему в FIFO `overtakes == 0`? 18
9.14 Почему при N=7 пропускная способность выше? 18
9.15 Какие ограничения у эксперимента? 18
10. СЛОВАРЬ
. 19
11. СВЕДЕНИЯ О РЕПОЗИТОРИИ
. 20

1. САМАЯ КОРОТКАЯ СУТЬ

Есть круглый стол, вокруг него сидят философы. Между соседями лежат вилки. Чтобы поесть, философу нужны **две** вилки — слева и справа. Философы по очереди думают, ждут и едят. Проблема в том, что вилок меньше, чем желающих, а философы работают одновременно.

В лабораторной работе нужно было:

- смоделировать философов потоками;
- не дать им одновременно взять одну и ту же вилку;
- не допустить взаимной блокировки;
- сделать центральную очередь заявок;
- реализовать два режима: `fifo` и `scan`;
- добавить тайм-ауты и аварийное завершение;
- проверить корректность и провести эксперименты;
- построить графики и оформить отчёт.

Главная идея решения: философ просит **сразу обе вилки**. Он либо получает обе, либо не получает ни одной. Поэтому он не может держать одну вилку и ждать вторую. Именно это убирает классический тупик.

2. ТЕОРИЯ БЕЗ СЛОЖНЫХ СЛОВ

2.1 Процесс и поток

Процесс — это запущенная программа со своей памятью. **Поток** — это «исполнитель» внутри процесса. У потоков одного процесса общая память: они видят одни и те же переменные.

В этой работе один процесс и много потоков:

- главный поток;
- N потоков-философов;
- один поток-watchdog, который следит за максимальным временем работы.

Потоки полезны, потому что позволяют делать несколько вещей одновременно. Но общая память — это и источник проблем: два потока могут одновременно изменить одну переменную, и результат будет непредсказуемым.

2.2 Гонка данных

Гонка данных (race condition) — ситуация, когда результат зависит от того, какой поток успел первым.

Пример без защиты:

```
Поток A: проверил, что вилка 0 свободна
Поток B: проверил, что вилка 0 свободна
Поток A: записал себя владельцем вилки 0
Поток B: записал себя владельцем вилки 0
```

Формально оба «проверили», но между проверкой и записью вклинился другой поток. В итоге одну вилку могут считать своей два философа. Это ошибка.

Чтобы её не было, проверку и изменение нужно делать как одну неделимую операцию. Для этого и нужна критическая секция.

2.3 Критическая секция и мьютекс

Критическая секция — участок кода, в котором одновременно может находиться только один поток. В C++ её обычно делают через `std::mutex`:

```
{
    std::lock_guard<std::mutex> lock(state.cs);
    // только один поток одновременно
    // проверяем вилки и назначаем владельцев
}
```

В работе один мьютекс `cs` защищает:

- очередь заявок;
- массив владельцев вилок `fork_owner`;
- состояния философов;
- статистику;
- проверку и изменение статусов заявок.

Почему один мьютекс, а не много маленьких? Так проще доказать корректность: все связанные данные меняются согласованно. Минус — меньше параллелизма, но для учебной модели это нормально.

2.4 Атомарная заявка на две вилки

Классическая ошибка в задаче о философах:

1. Взял левую вилку.
2. Ждёт правую.
3. Правую держит сосед, который ждёт свою вторую вилку.

Все ждут друг друга — это **взаимная блокировка** (deadlock).

В этой работе философ не берёт вилки по одной. Он создаёт заявку сразу на две вилки. Под мьютексом проверяется, что **обе** свободны. Если да — обе назначаются философу. Если нет — философ ждёт в очереди и не держит ничего.

Отсюда важное свойство: нет состояния «держу одну вилку и жду вторую». Значит, классический deadlock невозможен.

2.5 Семафор и событие

Семафор — объект со счётчиком. У нас счётчик равен 0 или 1.

- `sem_init(&event, 0, 0)` — создать семафор со значением 0;
- `sem_post(&event)` — увеличить значение, то есть подать сигнал;
- `sem_timedwait(&event, &deadline)` — ждать, пока значение станет больше нуля, но не дольше дедлайна.

Это похоже на auto-reset event в Windows:

- событие срабатывает один раз;
- после успешного ожидания семафор снова становится 0;
- если сигнал пришёл **до** ожидания, он не теряется: семафор уже равен 1, и `sem_timedwait` сразу вернётся.

Для каждой заявки создаётся свой семафор. Поэтому `sem_post` будит именно того философа, которому выдали вилки, а не всех сразу.

2.6 Тупик, голодание и справедливость

Deadlock (тупик) — все ждут ресурсы, которые никто не освободит. В нашей модели его нет из-за атомарной заявки на две вилки.

Starvation (голодание) — поток теоретически может ждать бесконечно долго, потому что другие постоянно обходят его. Это не тупик: система работает, но конкретный философ не получает ресурс.

Fairness (справедливость) — насколько равномерно ресурс достаётся участникам. FIFO ближе к справедливости, SCAN — к максимальной загрузке.

2.7 Пропускная способность и время ожидания

Пропускная способность (throughput) — сколько циклов еды выполняется в секунду. Чем больше, тем лучше используется ресурс.

Время ожидания (wait time) — сколько заявка стоит в очереди до выдачи вилок. Среднее ожидание показывает типичный опыт, максимальное — худший случай.

Бывает так, что алгоритм увеличивает пропускную способность, но ухудшает максимальное ожидание. Именно это и произошло с SCAN.

2.8 FIFO и SCAN

FIFO (First In, First Out) — «первым пришёл, первым обслужен». Смотрим только на голову очереди:

- если обе вилки головы свободны — выдаём;
- если нет — ждём, более поздние заявки не обходят.

Плюс: предсказуемость и отсутствие обходов. Минус: ресурс может простаивать, пока голова ждёт.

SCAN — просматриваем очередь и выдаём все заявки, которые сейчас можно удовлетворить. Если заявка была не первой, это **обход** (overtake).

Плюс: меньше простоя, выше пропускная способность. Минус: более поздние заявки могут обгонять ранние, максимальное ожидание растёт, возможно голодание.

3. ЧТО ТРЕБОВАЛОСЬ ПО ЗАДАНИЮ

Вариант 6: «Обедающие философы: справедливое распределение ресурсов».

Индивидуальные параметры:

- `STUDENT_ID = 28`;
- `SEED = 1000 + 97 * 28 = 3716`;
- `OPERATIONS = 100 + 5 * 28 = 240` циклов на философа;
- стандартная конфигурация: 5 философов по 100 циклов;
- индивидуальная конфигурация: 5 философов по 240 циклов.

Требовалось:

1. Реализовать N потоков-философов и N вилок.
2. Сделать запрос на вилки атомарным.
3. Использовать центральную очередь заявок.
4. Реализовать режимы `fifo` и `scan`.
5. Добавить тайм-аут ожидания.
6. Обеспечить корректное завершение и освобождение ресурсов.
7. Проверить инварианты.
8. Провести не менее трёх запусков для каждой конфигурации.
9. Измерить пропускную способность и время ожидания.
10. Построить графики и оформить отчёт.

4. КАК УСТРОЕНА ПРОГРАММА

4.1 Основные компоненты

Главный поток:
• разбирает аргументы командной строки;
• создаёт философов и watchdog;
• ждёт завершения потоков;
• печатает итоговую сводку;
• возвращает код завершения.
Поток-философ:
• думает;
• создаёт заявку на две вилки;
• ставит заявку в очередь;
• ждёт выдачи с тайм-аутом;
• ест;
• освобождает вилки;
• повторяет цикл.
Watchdog:
• периодически смотрит на время;
• если превышен `--max-duration-ms`, выставляет флаги остановки;
• философы завершаются аварийно, программа возвращает код 3.
Общие данные:
• `fork_owner[N]` — кто владеет каждой вилкой (`-1` — свободна);
• `queue` — центральная очередь заявок;
• `states[N]` — состояния философов;
• `stats[N]` — статистика;
• `waits` — времена ожидания для CSV.

4.2 Жизненный цикл одной заявки

Один цикл философа выглядит так:

```
THINKING
  -> создать Request на две вилки
  -> под mutex: WAITING, push в очередь, dispatch
  -> если можно — GRANTED, sem_post
  -> иначе sem_timedwait
  -> EATING
  -> под mutex: освободить вилки, dispatch
  -> снова THINKING
```

Если сработал тайм-аут:
• под тем же мьютексом проверяется статус заявки;
• если её уже успели выдать — философ ест;
• если нет — заявка удаляется из очереди и помечается `CANCELLED`;
• увеличивается счётчик тайм-аутов;
• вызывается `dispatch`, потому что очередь изменилась.
Проверка и отмена выполняются под одним мьютексом, поэтому не бывает ситуации «одновременно выдали и отменили».

4.3 Выдача вилок

Функция `dispatchLocked()` вызывается только под мьютексом. Она не создаёт отдельный поток-арбитр: решение принимается прямо в момент постановки заявки или освобождения вилок.

В FIFO:

```
пока очередь не пуста:
    смотрим на первую заявку
    если обе вилки свободны:
        выдать, удалить из очереди
    иначе:
        остановиться
```

В SCAN:

```
пройти по всей очереди:
    если обе вилки свободны:
        если заявка не первая — увеличить overtakes
        выдать, удалить из очереди
```

Именно поэтому в FIFO `overtakes == 0` всегда, а в SCAN обходы возможны.

4.4 Тайм-аут

Ожидание делается через `sem_timedwait`. Важно, что это **абсолютный** дедлайн: мы передаём момент времени, после которого ждать нельзя.

Если `sem_timedwait` вернул `ETIMEDOUT`, программа проверяет, не истёк ли общий дедлайн. Если истёк — заявка отменяется. Если нет — продолжает ждать.

Тайм-аут не означает «философ навсегда сломался». Он отменяет текущую попытку. Философ может позже начать новый цикл.

4.5 Watchdog

Watchdog — это страховка от зависания всей программы. Он не лечит взаимоблокировку, а аварийно завершает эксперимент, если тот идёт слишком долго.

При срабатывании:

- выставляется флаг `stop`;
- философы выходят из циклов;
- захваченные вилки освобождаются;
- программа печатает `STATUS=ABORTED`;
- код возврата — 3.

4.6 Инварианты

Инвариант — утверждение, которое должно быть истинным всегда. В работе проверяются:

1. Одна вилка не принадлежит двум философам.
2. Едящий философ владеет обеими своими вилками.
3. Число занятых вилок равно `2 * число едящих`.
4. В очереди не больше N заявок и нет дубликатов.
5. В FIFO нет обходов.
6. После завершения очередь пуста, вилки свободны, все циклы выполнены.

Если инвариант нарушен, программа увеличивает счётчик нарушений и завершается с ошибкой. В корректном запуске должно быть `INVARIANT_VIOLATIONS=0`.

5. ЧТО БЫЛО СДЕЛАНО ПО ЭТАПАМ

5.1 Этап 1. Проектирование

На этом этапе были созданы:

- структура каталогов `lab04-sync`;
- README с описанием варианта и параметров;
- `docs/architecture.md` с подробным описанием архитектуры;
- исходные диаграммы: архитектура, состояния, последовательности;
- заготовки для исходников, тестов, данных и отчёта.

То есть сначала была описана модель, а не сразу код. Это важно: так проще понять, какие данные общие, где нужна синхронизация и какие состояния есть у философа.

5.2 Этап 2. Базовая реализация

Были добавлены:

- `CMakeLists.txt`;
- `src/main.cpp`;
- разбор аргументов командной строки;
- состояния философов;
- центральная очередь;
- мьютекс и семафоры;
- режимы FIFO и SCAN;
- тайм-ауты;
- watchdog;
- статистика;
- журнал и CSV.

Фактически это ядро лабораторной: модель, которая умеет работать корректно и сохранять результаты.

5.3 Этап 3. Синхронизация и тестирование

Были добавлены автоматические проверки `tests/run_checks.py`, которые запускаются через CTest. Они проверяют:

- корректность FIFO;
- наличие обходов в SCAN там, где они ожидаются;
- тайм-ауты;
- превышение максимальной длительности;
- неверные аргументы;
- отсутствие нарушений инвариантов;
- корректность журнала и CSV.

Также отдельно проверены аварийные сценарии. Например, при `timeout=1 мс` и долгой еде программа сделала 2007 тайм-аутов, но все циклы завершились, а инварианты остались целы.

5.4 Этап 4. Эксперименты и отчёт

Были выполнены 18 запусков: 6 конфигураций по 3 раза.

Конфигурации:

- `standard_fifo`: 5 философов, 100 циклов, FIFO;
- `individual_fifo`: 5 философов, 240 циклов, FIFO;
- `individual_scan`: 5 философов, 240 циклов, SCAN;
- `load_n3_fifo`: 3 философа, 240 циклов, FIFO;
- `load_n5_fifo`: 5 философов, 240 циклов, FIFO;
- `load_n7_fifo`: 7 философов, 240 циклов, FIFO.

Результаты сохранены в `data/results.csv`, времена ожидания — в `data/waits.csv`, сырые журналы — в `data/raw/runs/`. По ним построены графики и подготовлен формальный отчёт.

6. КАК ЗАПУСКАТЬ И ПРОВЕРЯТЬ

Сборка:

```
cmake -S lab04-sync -B lab04-sync/build -DCMAKE_BUILD_TYPE=Release
cmake --build lab04-sync/build -j
```

Проверки:

```
ctest --test-dir lab04-sync/build --output-on-failure
```

Обычный запуск FIFO:

```
./lab04-sync/build/lab04_sync \
  --philosophers 5 \
  --cycles 240 \
  --seed 3716 \
  --mode fifo
```

Запуск SCAN:

```
./lab04-sync/build/lab04_sync \
  --philosophers 5 \
  --cycles 240 \
  --seed 3716 \
  --mode scan \
  --wait-csv lab04-sync/data/scan_waits.csv
```

Проверка тайм-аута:

```
./lab04-sync/build/lab04_sync \
  --philosophers 5 \
  --cycles 2 \
  --mode fifo \
  --timeout-ms 1 \
  --eat-min-ms 200 \
  --eat-max-ms 200
```

Проверка неверных аргументов:

```
./lab04-sync/build/lab04_sync --philosophers 1
```

7. РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ

Средние значения по трём запускам:

Таблица 1 – Средние результаты

Конфигурация	N × циклы	Режим	Время, с	Оп/с	Ср. ожид., мс	Макс. ожид., мс	Обходы
standard_fifo	5 × 100	fifo	8.67	57.7	54.3	84.3	0
individual_fifo	5 × 240	fifo	20.76	57.8	54.6	88.4	0
individual_scan	5 × 240	scan	18.26	65.7	43.2	193.8	280.7
load_n3_fifo	3 × 240	fifo	21.55	33.4	58.1	78.3	0
load_n5_fifo	5 × 240	fifo	20.70	58.0	54.3	86.5	0
load_n7_fifo	7 × 240	fifo	20.76	80.9	54.6	91.5	0

Во всех 18 запусках:

- STATUS=OK;
- INVARIANT_VIOLATIONS=0;
- TIMEOUTS=0;
- число завершённых циклов совпало с ожидаемым.

7.1 Почему FIFO даёт около 58 оп/с

В стандартной и индивидуальной конфигурациях FIFO пропускная способность почти одинаковая: 57.7 и 57.8 оп/с. Увеличение числа циклов в 5 раз увеличивает и время выполнения примерно в 5 раз, поэтому интенсивность не меняется.

FIFO не даёт ресурсу простаивать «просто так», но если голова очереди не может получить обе вилки, более поздние заявки не рассматриваются. Иногда это приводит к короткому простою, зато поведение предсказуемое.

7.2 Почему SCAN быстрее

SCAN показал 65.7 оп/с против 57.8 оп/с у FIFO — примерно на 13.7 % больше. Среднее ожидание снизилось с 54.6 до 43.2 мс.

Причина: SCAN не ждёт заблокированную голову. Если более поздняя заявка может быть удовлетворена, она получает вилки. Ресурс меньше простаивает, поэтому пропускная способность выше.

7.3 Почему у SCAN хуже максимальное ожидание

Максимальное ожидание выросло с 88.4 до 193.8 мс. Среднее число обходов — 280.7 за запуск.

Это плата за скорость: ранняя заявка может долго ждать, пока более поздние заявки обходят её. В худшем случае это переходит в голодание. Поэтому SCAN нельзя назвать справедливым без дополнительных ограничений.

7.4 Почему при большем N выше пропускная способность

При $N = 3$ одновременно может есть только один философ, при $N = 5$ — два, при $N = 7$ — три. Поэтому пропускная способность растёт: 33.4, 58.0 и 80.9 оп/с.

Среднее ожидание при FIFO остаётся близким к 54–58 мс, а максимальное — к 78–92 мс. Максимальная длина очереди равна $N - 1$: один ест, остальные ждут.

7.5 Графики

На рисунке 1 показана пропускная способность FIFO при разном числе философов.

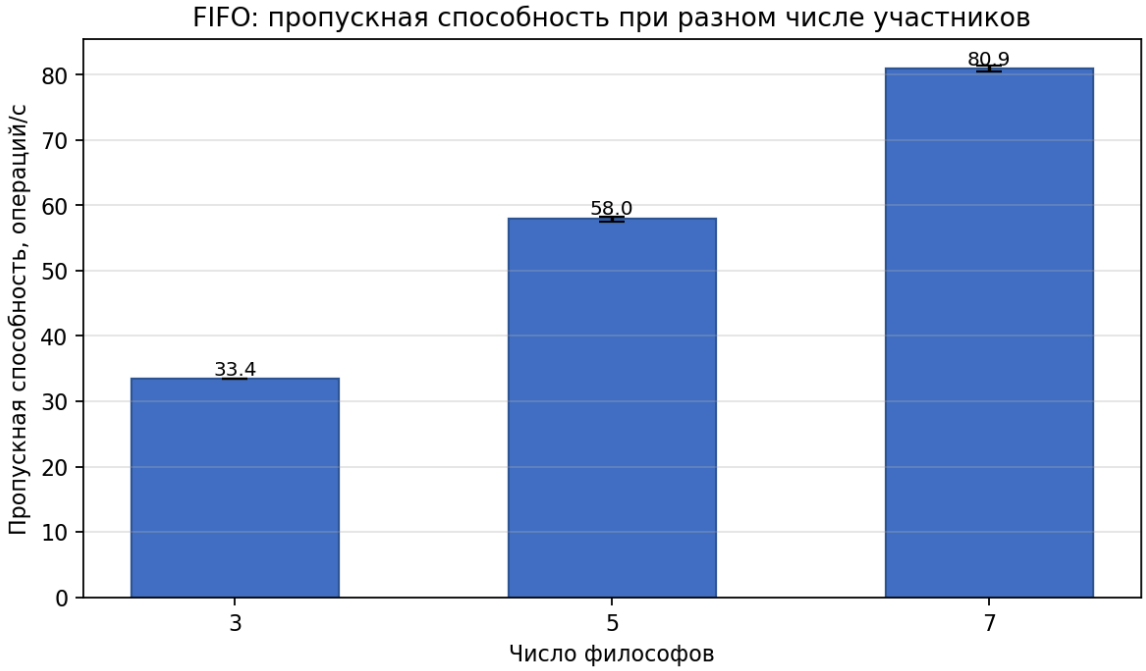


Рисунок 1 – Пропускная способность FIFO

На рисунке 2 — распределение времени ожидания по философам для FIFO и SCAN. У FIFO разброс уже и симметричнее, у SCAN есть длинные выбросы.

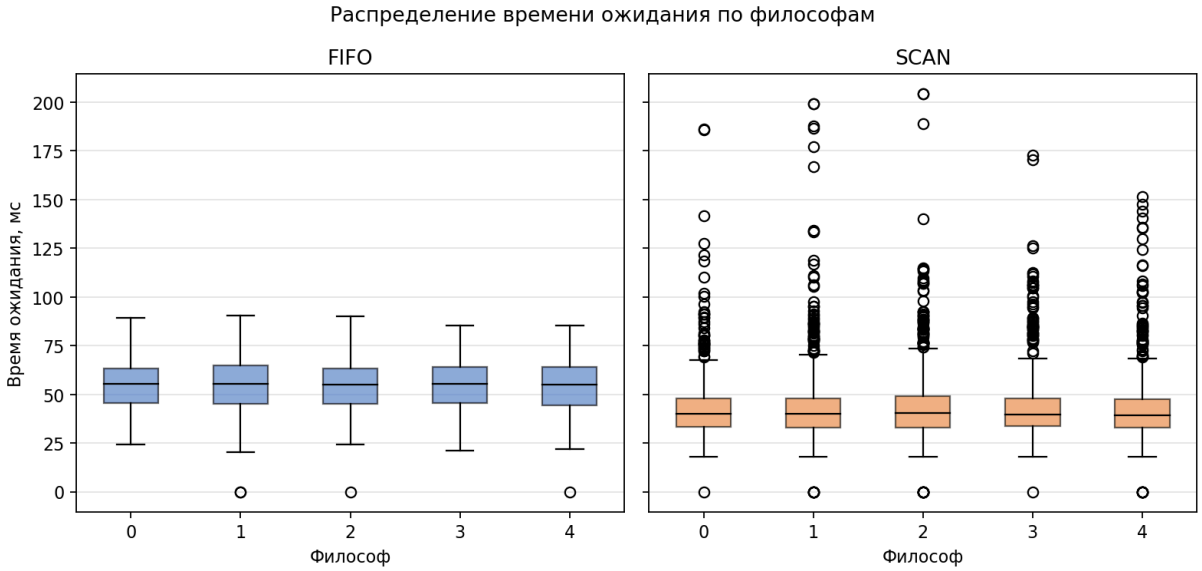


Рисунок 2 – Распределение времени ожидания

8. ЧТО БЫЛО БЫ, ЕСЛИ ДЕЛАТЬ С НУЛЯ

Если бы лабораторную нужно было делать с самого начала, порядок был бы таким.

1. **Понять модель.** Нарисовать стол, философов, вилки, состояния.
2. **Определить общие данные.** Очередь, владельцы вилок, состояния, статистика.
3. **Выбрать способ синхронизации.** Один мьютекс на все общие данные.
4. **Сделать атомарную заявку.** Сразу две вилки, без промежуточного владения.
5. **Сделать событие на заявку.** Отдельный семафор, чтобы будить только нужного.
6. **Реализовать FIFO.** Смотреть только на голову очереди.
7. **Реализовать SCAN.** Просматривать очередь и считать обходы.
8. **Добавить тайм-аут.** Отмена заявки без утечки ресурсов.
9. **Добавить watchdog.** Аварийное завершение по максимальному времени.
10. **Добавить инварианты.** Проверять корректность после каждой выдачи и в конце.
11. **Написать тесты.** Нормальные, тайм-аутные и аварийные сценарии.
12. **Провести эксперименты.** Не менее трёх запусков на конфигурацию.
13. **Построить графики и объяснить результаты.** Не просто числа, а причины.
14. **Оформить отчёт.** Схемы, таблицы, код, выводы.

9. ВОПРОСЫ НА ЗАЩИТЕ

9.1 Зачем заявка сразу на две вилки?

Чтобы философ не мог держать одну вилку и ждать вторую. Это исключает классическую взаимоблокировку.

9.2 Почему очередь центральная?

Так проще управлять порядком обслуживания и реализовать FIFO/SCAN. Все заявки находятся в одном месте и защищены одним мьютексом.

9.3 Зачем отдельный семафор на каждую заявку?

Чтобы `sem_post` будил только того философа, которому выдали вилки. Если использовать один общий семафор, придётся будить всех и заново проверять условия.

9.4 Почему сигнал не теряется?

Семафор хранит значение 1. Если `sem_post` вызван до `sem_timedwait`, следующее ожидание сразу завершится успешно. Это и есть свойство auto-reset event.

9.5 Что такое overtake?

Это ситуация, когда заявка, стоящая в очереди позже, получает вилки раньше более ранней заблокированной заявки. В FIFO такого быть не должно, в SCAN — может быть.

9.6 Почему FIFO справедливый?

Он обслуживает только голову очереди. Пока голова не может получить вилки, никто её не обходит. Поэтому нет голодания из-за обгонов.

9.7 Почему SCAN быстрее?

Он не ждёт заблокированную голову и отдаёт вилки тем, кто может их использовать. Меньше простоя — выше пропускная способность.

9.8 Почему у SCAN хуже максимальное ожидание?

Потому что ранние заявки могут многократно обгоняться. Среднее ожидание может быть ниже, а худший случай — значительно хуже.

9.9 Как проверяется отсутствие гонок?

Все проверки и изменения общих данных выполняются под одним мьютексом. Дополнительно программа проверяет инварианты после выдачи, освобождения и в конце. Тесты запускают разные режимы и аварийные сценарии.

9.10 Что будет при тайм-ауте?

Заявка отменяется, если её ещё не выдали. Философ не держит вилки, поэтому освобождать нечего. После отмены вызывается `dispatch`, чтобы очередь продолжила работу.

9.11 Зачем watchdog, если есть тайм-ауты?

Тайм-аут ограничивает ожидание одной заявки. Watchdog ограничивает время всей программы. Это разные уровни защиты.

9.12 Почему нет отдельного потока-арбитра?

Решение о выдаче принимается под мьютексом в момент постановки заявки или освобождения вилок. Отдельный поток не нужен: он только усложнил бы синхронизацию.

9.13 Почему в FIFO `overtakes == 0`?

Потому что FIFO не рассматривает заявки после заблокированной головы. Значит, обойти голову невозможно.

9.14 Почему при N=7 пропускная способность выше?

Одновременно могут есть три философа вместо двух при N=5 и одного при N=3. Больше параллельных едоков — больше завершённых циклов в секунду.

9.15 Какие ограничения у эксперимента?

Измерения на одной машине, по три запуска, небольшие длительности имитации. Результаты зависят от планировщика ОС и фоновой нагрузки. Для строгих выводов нужно больше повторов и изоляция ядер.

10. СЛОВАРЬ

- **Поток** — исполнитель внутри процесса, имеет общую память с другими потоками.
- **Мьютекс** — примитив взаимного исключения.
- **Критическая секция** — код, выполняемый только одним потоком.
- **Семафор** — счётчик с операциями ожидания и сигнала.
- **Event** — событие, которое может быть в сигнальном состоянии.
- **Deadlock** — взаимная блокировка.
- **Starvation** — голодание.
- **Fairness** — справедливость распределения.
- **Throughput** — пропускная способность.
- **Инвариант** — утверждение, которое всегда должно быть истинным.
- **Watchdog** — сторожевой механизм аварийного завершения.

11. СВЕДЕНИЯ О РЕПОЗИТОРИИ

Таблица 2 – Сведения о репозитории

Параметр	**Значение**
Ссылка на репозиторий	https://github.com/Dima-Vasilenko/osisp
Хеш окончательного коммита	H4_3
Каталог лабораторной работы	`ossp-labs/lab04-sync`
Этап 1. Проектирование	H1 — структура проекта, README, архитектура, диаграммы, скелет
Этап 2. Базовая реализация	H2 — CMake, `src/main.cpp`, CLI, состояния, центральная очередь
Этап 3. Синхронизация и тестирование	H3 — `tests/run_checks.py`, CTest, инварианты, тайм-ауты, аварийные режимы
Этап 4. Эксперименты и отчёт	H4_3 — 18 запусков, `data/results.csv`, графики, `report.pdf`, `explained.pdf`

Главный вывод. Работа — это не про «философов» как таковых. Это про то, как несколько потоков делят ограниченный ресурс и как доказать, что при этом нет гонок, тупиков и утечек. Задача о философах просто делает эти проблемы наглядными.